

哈尔滨工业大学（深圳）2023 年秋季学期

高级语言程序设计期末试题（A）（回忆版）参考答案与解析

解析撰写：Zjl37

审阅：Origami404

✧ 答案速查

一、选择题

1	2	3	4	5	6	7	8	9	10
C	A	C	A	B	A	C	B	C	没有正确答案
11	12	13	14	15	16	17	18	19	20
A	A	B	D	D	A	D	B	A	A
21	22	23	24	25					
B	A	C	B	A					

二、程序填空题

答案不唯一。

- `6n`
 - `return 1;`
 - `sum = 0`
 - `sum += p[i]`
- `'\0'`
 - `x`
 - `z + 1`
 - `num`
- `struct student`
 - `strcmp(stu[i].name, str) == 0`
 - `break`
- `"%d", &a[n]`
 - `a[0]`
 - `0`
 - `n`
- `ch >= 'A' && ch <= 'Z' || ch >= 'a' && ch <= 'z'`
 - `ch >= '0' && ch <= '9'`
 - `ch == ' '`

✧ 解答提示

一、选择题

1 C do-while 循环是先执行循环体语句，后判断表达式；而 for 循环是先判断表达式再执行循环体语句。

2 A 在 C 语言中，表达式 `'0' <= c <= '9'` 不会像数学记号一样的意思被理解，而会被理解为 `('0' <= c) <= '9'`（因为小于等于运算符结合性为从左到右）。'0' 先与 'A' 作比较，因为 '0' 和 'A' 的 ASCII 值分别为 48、65，比较得真（1）；1 再与 ASCII 值为 57 的 '9' 作比较，得真（1）。因此程序进入第一个分支，输出 YES。若想要判断字符 c 是否为数字，应当写 `'0' <= c && c <= '9'`。

注：有些其他编程语言，如 python，对这种语法作了特殊支持，称为“连续比较”。在 python 中，表达式 `'0' <= 'A' <= '9'` 等价于 `'0' <= 'A' and 'A' <= '9'`，其值为 `False`。

Tip: 解答本题时，不记得 `'0'` 和 `'A'` 的 ASCII 值也没关系，比较的结果要么是 0 要么是 1，都比在可见区段的 ASCII 字符 `'9'` 小。

3 C A 选项，试图用长度为 8 的字符串初始化大小为 8 的字符数组，但字符串末尾还需要一空终止符 (`'\0'`)，放不下；B 选项，初始化列表长度为 6，但数组大小为 5，放不下；D 选项，表示数组大小的下标只有第一维可以省略。（否则编译器不知道如何布局内存。）

4 A 联合体（共用体，union）的成员的内存是重叠的，其大小只需要达到其最大成员的大小就能储存所有成员。`float` 类型一般遵循 IEEE-754 binary32 格式，占 32 位（bit）；`int` 类型至少有 16 位，在现代绝大多数电脑上为 32 位；`char` 类型占 1 字节（byte）。1 字节有 8 位，`sizeof(test)` 告诉我们这个联合体在内存中占多少字节，因此，其最有可能的结果为 $32 \div 8 = 4$ 。

注：一字节是多少比特？

严格地说，1 字节不一定是 8 位。在今天，几乎所有的计算机体系结构都采用了 8 比特的字节，然而历史上也存在一些使用其他字节大小的计算机体系结构。C 语言标准规定一字节所含比特数 `CHAR_BIT` 至少为 8。感兴趣的同学可以阅读[此讨论贴](#)了解更多。

5 B `n++` 是后置自增运算，其结果是增加前 `n` 的值，即 9， $9 < 10$ ，进入第一个分支，程序输出 `n` 的值，这时为 10。

Tip: 可以这样记忆自增/自减运算符的行为：

前缀自增如 `++a`，“++”在前，先增加，后返回。

后缀自增如 `a++`，“++”在后，先返回，后增加。

6 A 程序首先将 `main` 函数中的局部变量 `y` 的地址、`x` 的地址依次传给 `fun` 函数，作为其 `x`、`y` 指针参数，因此在 `fun` 函数中首先依次输出参数 `x` 指向的值和 `y` 指向的值，即 `main` 函数中的 `y` 和 `x`，即 2 1，然后分别将 `fun` 的参数 `x`、`y` 指向的值修改为 3、4，即将 `main` 函数中的 `y` 修改为 3，将 `main` 函数中的 `x` 修改为 4。于是程序返回到 `main` 函数后，依次输出 `x` 和 `y` 的值为 4 3。因此，程序的运行结果是 2 14 3，选择最佳答案 A。

7 C 程序首先定义并初始化了长度为 10 的 `char` 型数组 `a`，并定义了指针 `p` 指向 `a` 下标为 5 的位置（“4”所在的位置），然后通过 `printf` 语句，用 `"%d"` 占位符指示将 `--p` 指向的值当作整数输出。`--p` 的值是指针向左移动一位后的值，即 `a` 下标为 4 位置的地址（即 `a + 4` 或 `&a[4]`），指向 `a` 中的“5”。

8 B 在计算机中， n 位无符号整数的取值范围为 $0 \sim 2^n - 1$ 。unsigned short int 与 short int 占用相同的大小，由题意为 2 字节，即 16 位，故其可存储的值取值范围为 $0 \sim 65535$ 。

9 C 程序用 for 循环执行两次 `k = f(a++)`；该语句调用了函数 `f` 并将其返回值赋给 `k`。（由于 `f` 的函数体里未使用参数 `m`，其值并不重要。）在 `f` 的函数体的 `a = c++`，`b++`；语句中，由于逗号的优先级低于赋值运算，程序分别对表达式 `a = c++` 和 `b++` 求值，前者将 `c` 自增 1 并将其增加前的值赋值给 `f` 中定义的局部变量 `a`。（由于 `b` 仅被修改未被使用，其值亦不重要。）因此，第一次调用 `f`，将 `c` 增加至 4 而返回 3；（其实这一次的返回值亦不重要，因为 `k` 会被第二次调用 `f` 的返回值覆盖。）第二次调用 `f` 时，由于 `c` 是静态（static）变量，其值仍为 4，该值先后被赋给 `f` 的局部变量 `a`，从 `f` 返回，赋值给 `k`，最后被输出。

注：static 关键字可以指定变量为静态存储期（storage duration）。拥有静态存储期的变量，其生存期（lifetime）为从程序开始到程序结束；在整个程序的执行过程里，只会在 main 函数之前初始化一次存储于其中的值。也就是说，在函数中定义的静态变量，不会在每次函数调用中被（重新）初始化。

10 - 程序 main 函数中用 for 循环两次调用 Fun 函数并输出其返回值。

- 第一次调用 Fun 传入的 `m` 参数为 4，`m /= 2`；`m = m * 2`；两条语句将 `m` 变为不大于它的最大偶数，执行完后 `m` 的值为 4，不为零，因此 if 进入第一个分支，将 `n` 乘以 `m`，`n` 变为 4，再递归调用并返回 Fun(2)；
 - 这次调用中 if 条件判断时 `m` 的值为 2，仍进入第一个分支，将 `n` 乘以 2。由于 `n` 是静态（static）变量，其值 `n` 从刚才的 4 变为 8。然后递归调用并返回 Fun(0)；
 - 这次调用中 if 条件判断时 `m` 的值为 0，进入 else 分支，返回 `n` 的值，即 8。该值被一路返回到 main 中并输出。
- 第二次从 main 中调用 Fun(5)，Fun 中 if 条件判断时 `m` 的值已变为 4，进入第一个分支，将 `n` 乘以 4，`n` 从 8 变为 32，然后递归调用并返回 Fun(2)；
 - 这次调用中 if 条件判断时 `m` 的值为 2，仍进入第一个分支，将 `n` 乘以 2。`n` 从 32 变为 64。然后递归调用并返回 Fun(0)；
 - 这次调用中 if 条件判断时 `m` 的值为 0，进入 else 分支，返回 `n` 的值，即 8。该值被一路返回到 main 中并输出。

8

综上所述，程序的运行结果是 64，没有正确答案。

11 A 注意到 fun 是纯函数，即其返回值仅取决于参数（对于同样的参数作为输入，返回值总是相同的。），且不对外部产生任何副作用。因此，它在适当的取值范围内等同于数学函数
$$\text{fun}(n) = \begin{cases} 2 & \text{if } n=1 \text{ or } n=2 \\ n - \text{fun}(n-1) & \text{otherwise} \end{cases}$$
。所以，程序的输出是 $\text{fun}(3) = 3 - \text{fun}(2) = 3 - 2 = 1$ 。

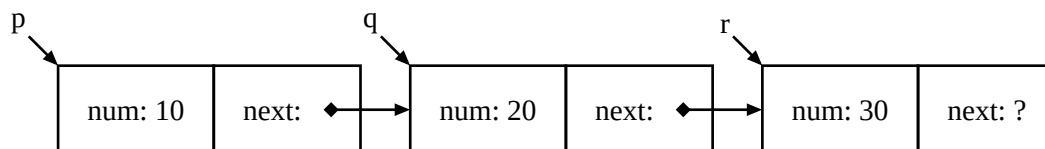
12 A 分析 `amovep` 函数，可知其作用是将二维数组 `a` 的内容逐行连续存入以 `p` 为开始的内存区域，即 `a[i][j]` 被存入 `p + (i*3 + j)`（或 `6p[i*3 + j]`）位置，示意图如下（`p` 表示 `main` 函数中的 `p`）。故 `p[2]` 得到 `a[0][2]` 的值，为 5；`p[5]` 得到 `a[1][2]` 的值，为 6。

地址	<code>p</code>	<code>p + 1</code>	<code>p + 2</code>	<code>p + 3</code>	<code>p + 4</code>	<code>p + 5</code>	<code>p + 6</code>	<code>p + 7</code>	<code>p + 8</code>
int 值	<code>a[0][0]</code> 即 1	<code>a[0][1]</code> 即 3	<code>a[0][2]</code> 即 5	<code>a[1][0]</code> 即 2	<code>a[1][1]</code> 即 4	<code>a[1][2]</code> 即 6	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>

13 B 定义在 `<stdio.h>` 中的标准库函数 `fopen` 的第二个参数指定了打开文件的模式。当其为 `"w"`（写）时，若文件不存在则创建新文件，若文件已存在则销毁内容。因此，只有第二次写入的内容留在文件中。

14 D 程序传给 `fun` 的第一个参数为 `char` 型变量 `b` 的地址，而第二个参数按值传递。在 `fun` 中，`*c = *c + 1`，`d = d + 1`；语句将 `c` 指向的值（即 `main` 中 `b` 的值）增加 1（从 `'a'` 变为 `'b'`），并将 `fun` 函数局部的 `d` 增加 1。（从传入的 `'A'` 变为 `'B'`）（请注意，`main` 中的 `a` 值未改变。）随后此二值被依次输出。返回到 `main` 后，`a` 从来未被修改过，其值仍为 `'A'`，`b` 的值已被修改为 `'b'`，这二值被依次输出。

15 D `Node` 结构体由一 `int` 型成员和一 `Node` 的指针构成，可以将 `Node` 看成链表中的节点，则 `num` 成员表示该节点储存的数据值，`next` 为指向下一个节点的指针。程序在 `main` 函数中分配了 3 个节点并设置了它们的数据值和先后关系。至 `printf` 之前，链表的结构示意图如下：



故最后 `p->num + q->next->num` 即为 `10 + r->num`，即 `10 + 30 = 40`。

16 A 在 C 语言中，花括号“`{}`”还可以用来包围复合语句（块）、初始化列表，声明结构体等。事实上，函数体就是一条复合语句。B 项错误。函数名须是标识符，且不得是保留标识符（如关键字，与标准库函数相同的名字等）。C 项错误。分号是语句（一般指表达式语句，也适用于 `do-while` 循环语句和跳转语句）的一部分，用于表示语句的结束。D 项错误。

17 D 每个复合语句都界定了一个块作用域，其中定义的变量只在该块内可见，在复合语句花括号外部无法访问。故 D 选项说法不准确。A、B、C 项皆正确，选择最佳答案 D。

18 B 程序中定义的 `struct s data[2]` 是含有两个结构体 `s` 的数组，它是如何被初始化的呢？嵌套的初始化列表的内层花括号在一定条件下可以省略，此时会先从列表中取足够的（对于本题是 2 个）元素（10 和 100）来初始化前面的数组元素（`data[0]`），后面的列表元素再留给后面的数组元素。所以本题中的初始化列表就等同于

`{ { .x = 10, .y = 100 }, { .x = 20, .y = 200 } }`。当 `data` 被赋值给 `p` 时，其从数组类型

(`struct s [2]`) 退化为指针类型 (`struct s *`)，值为数组首元素地址，故 `p->x` 即为 `data[0].x`，为 10；`++(p->x)` 的值为其增加后的值，为 11。

19 A 请注意，出现在 `while` 的条件表达式中的单个等号是赋值运算符，并且整个赋值表达式 (`*p2 = *p1`) 会返回左边被修改后的值 (这里即 `*p1`)。因此，在 `while` 的每一次循环体执行之前，在对条件表达式求值时就会将 `p1` 指向的字符赋值给 `p2` 指向的字符，若其非零 (还未到 `p1` 所指的字符串结尾)，则执行循环体，即同时将 `p1`、`p2` 后移一位。这样直到 `p1 == '\0'` 时，已经将 `p1` 所指的字符串 (包括其末尾的 `'\0'`) 复制到 `p2` 所指的内存空间了。

20 A `scanf` 中用 `"%d"` 占位符指示读入 `int` 型整数时，相应位置参数接受的类型应为 (`int *`) (指向一个 `int` 的指针)，`pa` 正好符合。`*pa` 对 `pa` 解引用以访问 `a`，`*pa` 与 `a` 类型都是 `int`；`&pa` 取 `pa` 的地址，得指向 `pa` 的指针，其类型为 (`int **`)。

21 B `swap1` 函数接受一个整型数组，并交换数组中第一个和第二个元素的值。当数组作为参数传递给函数时，实际上传递的是数组的地址 (或者说数组的首元素的地址)，通过 `c` 可以访问到 `main` 中的 `a`。因此调用 `swap1(a)` 会将数组 `a` 中的元素进行交换，最终数组 `a` 的值变为 {5, 3}。而 `swap2` 函数接受两个整型参数，并交换这两个参数的值。调用 `swap2(b[0], b[1])` 只是将 `b[0]` 和 `b[1]` 的值传递给了 `swap2` 函数，而在函数内部进行的交换并没有影响到数组 `b` 的值。因此，数组 `b` 的值保持不变，仍为 {3, 5}。

22 A 程序中定义的 `char *p[10]` 为含有 10 个 (`char *`) (字符指针) 的数组，其中前五个已被初始化指向内存中相应的字符串。`p[4]` 即是 `"cd"`，用 `strlen` 求得其长度为 2。

Tip: 如何阅读复杂声明

C 语言的“如何使用，如何声明”原则可以帮助我们理解复杂的类型与声明。例如，要使用本题定义的 `char *p[10]` 得到一个基本的 `char` 类型，只需对与声明符形式相同的表达式 `*p[0]` 求值，即先对 `p` 取下标，再解引用。(注：后缀运算符 `[]` 优先级高于前缀运算符 `*`) 因为要先对 `p` 取下标，所以 `p` 首先是一个数组；其中的元素需要被解引用，故为指针；最后得到一个 `char`，故 `p` 是一个字符指针数组。

又如，第 12 题中 `amovep` 函数接受参数 `int (*a)[3]`，要使用之，从中间的标识符 `a` 开始，首先需对其解引用，故 `a` 首先是个指针；然后要取下标，故 `a` 指向的对象是数组；然后就能得到 `int`，故数组中的元素是整数。综上，`a` 是一个整型数组指针。它可由一整型二维数组退化而来。

以上阅读声明时“`[]` 对应数组、`*` 对应指针”的使用方法仅仅是形式和技巧。真正使用这些变量时，也可以对指针取下标 (表达式 `a[i]` 其实等同于 `*(a + i)`)，也可以对数组解引用 (得其首个元素)。

思考题

相信你已经理解了声明的基本语法，下面试着解读一个这个定义吧：

```
int (*(*foo)(double))[3] = NULL;
```

答案在[此网页](#)上。

23 C 请注意，程序的 switch 语句体中没有任何 break 语句，会发生“fall-through”，即当前 case 标号后，下一个标号前的语句执行完后，不会跳出 switch 语句体，而继续向下执行其他标号下的语句。故当 i 等于 0 时，从 case 0: 进入 switch，会输出 3 遍；当 i 为 1 时，不匹配任何 case 标号的值，从 default: 进入 switch，输出 1 遍；当 i 为 2 时，从 case 2: 进入 switch，输出两遍。

24 B 本题要求计算 5 的阶乘。B 项在每次 for 循环体的开头都将 p（重新）赋值为 1，无法记录下累乘的积。

25 A 由于 p 是 (int *) 型指针，只能对其解一次引用或取一次下标，排除 B、D 选项。一般按行存储二维数组，即同一行的 n 个元素在内存中相邻，一行后面紧跟着下一行。将 (int *) 型指针向后移 n 个位置，就是移动到同一列的下一行，故下标中 i 的系数为 n，j 的系数为 1。

二、程序填空题

1

(1) 由前一条语句的提示，这里应该输入数组大小（学生数量）；由下面 malloc 中用到了 n，故 n 应表示数组大小。填 &n。不要忘记取地址符。

(2) 这里程序申请不到足够的内存，不能继续正常执行，应当退出。使用 return 语句使 main 函数返回。为了指示程序异常退出的状态，最好返回 EXIT_FAILURE 等非零值。也可调用定义于 <stdlib.h> 中的 exit 函数退出程序。

(3) 需要初始化 sum = 0，后面才能使用 sum

(4) 每次循环将一个同学的成绩累加到 sum 中，填 sum += p[i]

2

这个算法的思路是：枚举 str 串每个位置 x 作为起始位置，逐个字符地比较其后的部分是否与 substr 相等，若是则将计数器 num 增一。

外层循环从 0 开始递增枚举 x，循环条件为 x 未到达 str 的末尾位置，即 str[x] 非零，第 (1) 空可以填 '\0'。

内层循环使用了两个变量表示待比较字符的位置。根据已给出的循环条件

`str[y] == substr[z]` 可知 `y` 为 `str` 串的下标, `z` 是 `substr` 的下标。`z` 从 0 开始, 相应地 `y` 应该从 `x` 开始。第 (2) 空填 `x`。

在内层 `for` 循环的每次循环体中, `substr` 的 `z` 位置及之前的字符已经被逐一被比较过跟 `str` 中的相等了。此时若 `z` 下一个位置处是空终止符 `'\0'`, 则说明整个 `substr` 都被比过了, 与 `str` 的 `x ~ x + strlen(substr)` (右端不含) 位置部分相等, 将计数器 `num` 增一并跳出内层循环。因此第 (3) 空可以填 `z + 1`。

最后, 第 (4) 空填 `num`, 将出现次数返回给调用方。

3

(1) 要定义一个元素为 `student` 结构体的数组, 使用的类型说明符为 `struct student`

注: 不同于 C++ 语言, 这里的 `struct` 关键字不能被省略。

(2) 这里用定义在 `<string.h>` 中的 `strcmp` 函数可以判断用户输入的名字与记录中某个学生名字是否相同。当传给 `strcmp` 函数的两个字符串相等则会返回零。

Tip: 你可以看到程序末尾也用了 `strcmp` 函数来判断用户输入是否是 0。这提示了 `strcmp` 的用法。

(3) 当找到指定名字的学生后就可以 `break` 跳出, 不必再比较数组后面的学生。

4

(1) 用 `"%d"` 占位符指示读入 `int` 型的整数, 同时相应地传入指针参数 `&a[n]` (或 `a + n`), 前一种写法不要忘记取地址符。

(2)(3)(4) 容易看出, `max` 是记录最大值的变量, `pos` 是记录最大值所在位置的变量。当 `for` 循环中枚举到一个比当前最大值 `max` 还要大的数时, 则需用该数更新 `max` 并相应地设置 `pos`。故第 (4) 空填 `a[n]`。在给 `max` 和 `pos` 赋初始值时, 要保证 `max` 不会比最终真实的最大值大, 且即使 `max` 和 `pos` 未被更新, `pos` 表示的位置也是正确的。因此, 可以选定一个 0 到 9 中的整数设为 `pos` 的初始值, 同时将 `max` 初始值设为数组 `a` 该位置上的元素值。

5

这个程序没有包含 `<ctype.h>` 头文件, 不能使用 `isalpha` 等标准库函数来检查字符类别。可以通过比较字符的 ASCII 值来实现——要知道大写字母、小写字母、数字字符在 ASCII 码表中各占据一块连续的位置, 且字母按字母表顺序, 数字按 0 ~ 9 顺序排列。解答本题亦不必记住它们具体的 ASCII 值, 只需用 `'A'` 等字面量指代。

不要写出像选择题第 2 题中的“连续不等式”。在纸上作答时, 建议将 `==` 写清晰使得阅卷者看得出为两个等号, 不要写“`≤`”等符号。

三、编程题

设 $A = (a_{ij})_{M \times N}$, $B = (b_{ij})_{N \times M}$, $C = (c_{ij})_{M \times M}$, 矩阵乘法 $C = AB$ 的公式是

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{iN}b_{Nj} = \sum_{k=1}^N a_{ik}b_{kj},$$
$$i, j = 1, 2, \dots, M.$$

要计算矩阵 C 中的每一个元素, 可先用双层循环枚举其两个维度下标; 要计算求和, 还需要一层循环以枚举变量 k . 事实上, 这三层循环可以以任意顺序嵌套, 只要保证首次向二维数组 c 的元素累加前, 该元素已被初始化为零。

有的同学可能会想调用定义于 `<string.h>` 中的 `memset` 函数给二维数组 c 清零, 请注意

`memset(c, 0, sizeof c);` 是不对的, 因为作为函数参数的 `int c[M][M]` 实际上是数组指针, 在该函数中 `sizeof c` 是当前平台指针的大小, 不是二维数组 c 的大小。可以写

`memset(c, 0, sizeof(int)*M*M);`

不要忘记在读取用户输入前输出题目规定的提示信息。

程序的具体实现写法是多样的, 只要满足题目要求、能通过编译且正确实现功能即对。以下是一份参考程序:

```
#include <stdio.h>

#define M 3
#define N 4

void MultiplyMatrix(int a[M][N], int b[N][M], int c[M][M])
{
    for (int i = 0; i < M; ++i)
    {
        for (int j = 0; j < M; ++j)
        {
            c[i][j] = 0;
            for (int k = 0; k < N; ++k)
            {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

void InputMatrixA(int a[M][N])
{
    puts("Input 3*4 matrix a:");
    for (int i = 0; i < M; ++i)
    {
        for (int j = 0; j < N; ++j)
        {
            printf("a[%d,%d]: ", i, j);
            scanf("%d", &a[i][j]);
        }
    }
}
```



```

    }
}

void InputMatrixB(int b[N][M])
{
    puts("Input 4*3 matrix b:");
    for (int i = 0; i < N; ++i)
    {
        for (int j = 0; j < M; ++j)
        {
            printf("b[%d,%d]: ", i, j);
            scanf("%d", &b[i][j]);
        }
    }
}

void OutputMatrix(int a[M][M])
{
    puts("Results:");
    for(int i = 0; i < M; ++i) {
        for(int j = 0; j < M; ++j) {
            printf("\t%d", a[i][j]);
        }
        printf("\n");
    }
}

int main()
{
    int a[M][N], b[N][M], c[M][M];
    InputMatrixA(a);
    InputMatrixB(b);
    MultiplyMatrix(a, b, c);
    OutputMatrix(c);
    return 0;
}

```

✧ 校记

本回忆版试卷与考试时看到的原卷有一些不同，了解以下细节可能会对你有额外帮助。

- 为了美观易读，本卷所有代码片段经过格式化，使用等宽字体 Fira Mono 排版并启用代码高亮。而原卷全为黑白，代码片段使用普通西文衬线体排印，无明显样式区别于周围文本段落，且代码风格更紧凑。
- 选择题第 7 题代码中的双引号，原卷印为两边形状不一样的弯引号，形如 “ ”。还有其他几处单、双引号如此。
- 选择题第 7、12 题代码中的 `#include` 预处理器行，原无。
- 原卷许多程序中函数定义前省略返回值类型，不符合 C99 以来的标准，径添。
- 选择题第 13 题代码中的 `fclose(fp);` 语句，原缺。

注：选择题第 13 题程序若不使用 `fclose` 关闭文件，将会造成同一文件打开两次。根据 C 语言标准，这是实现定义行为（*implementation-defined behavior*）。经我们的测试，在 Windows 平台上（`msvc`，`mingw-w64 gcc`），文件 `test` 最终的内容是 `helloorld`；在 Linux 平台上（`gcc`，`clang`），文件 `test` 最终的内容是 `new world`。

- 程序填空题第 2 题代码中的 `break;` 语句，原缺。

注：程序填空题第 2 题若无 `break` 语句，对于 `substr` 是 `str` 的后缀的情况，如 `check("babceabceabcd", "abcd")`，程序会非法越界访问内存。想一想为什么。